**What are Operators in C?**

Operators can be defined as the symbols that help us to perform specific mathematical, relational, bitwise, conditional, or logical computations on operands. For example, ‘+’ is an operator used for addition, as shown below

```
a +  b;
```

Here, ‘+’ is the operator known as the addition operator, and ‘a’ and ‘b’ are operands.

The addition operator tells the compiler to add both of the operands ‘a’ and ‘b’

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

Types of Operators in C

categorised based on operands and based on operation

Bases on number of operands

1:Unary:only one operand

2:Binary:two operand

3:Ternary:three operand

Based onTypes of operation

1:Arithmetic Operators

2;Relational Operators

3.Logical Operators

4. Bitwise Operators

5.Assignment Operators

6...Other Operators

**Unary Operators**:

Operators that operate or work with a single operand are unary operators.

example: Increment(++) and Decrement(–-) Operators

++, --, sizeof, *, !,+ ,-

////////////////////////////////////////////////////////////////////////////////////////////////////

Binary operator

Operators that operate or work with two operands are binary operators.

Assignment-----> =

Arithmetic--------> +, - ,*, /, %

Logical----------->&& ,||

bitwise----------->& ,|, ^ ,<<, >>

relational-------> <,>, <=,>==,==,!=

compound arithmetic--->+=, -== , &=, *=,.....

Teranary--------> ? : ;

////////////////////////////////////////////////////////////////////////////////////////////////////

int x=10;

y=20;

x= y; ////Assignment//valid

y = x = 30;///valid

printf(“%d” %d “, x, y);

////////////////////////////////////////////////////////////////////////////////////////////////////

x=y;//valid

10=y;//invalid

x=y+z ;///valid

y+z =x;///invalid

variable=  var

constant

expression

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

y=30=x = 40;/////wrong

assignment from right to left

precedence and associativity comes into role

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

Arithmetic operator

* /  % ---------->Associativity left to right

+  - ------>left to right

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

10 +20 * 30

----------> can be 30 * 30 =900/////wrong

( 10  +(20 *30 ))= 10 + 600=610

Here compiler follows code compilation during code compilation

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

Example

x=10, y= 20  ,z =30;

x*y +z ===>10 *20 + 30

*--priority is first

+---second priority

so (10  *20) +30;

200 +30=230;

////////////////////////////////////////////////////////////////////////////////////////////////////

2:x +y * z

////////////////////////////////////////////////////////////////////////////////////////////////////

Q: x= 10, y=10, z=10

x/z + y*x -z %10 *y

(x/z) + y*x -z %10 *y .....(+  /  %)....>same priority...so associativity -left to right

(x/z) + (y*x) -z%10 *y

(x/z) +(y*x) -(z%10)* y

(x/z) +(y*x) -( (z%10) * y)

( (x/z) + (y*x) ) - ( (z %10) * y) )

( (10/10)+(10*10)) -((10%10)*10))

(1+100)-(0 *10)

101- 0=101

EXAMPLE

b=a *b/a=b;///invalid/

b is assigned to expression ,invalid

a=10,b=20

(a*b)/(a=b)///valid

////////////////////////////////////////////////////////////////////////////////////////////////////////////

Relational operator

< <= > .=

== !=

////////////////////////////////////////////////////////////////////////////////////////////////////////////

int x=10,y=20,z=30

int result;

result=x !=y;///result =1;

result= z> y >x ////result=0;

q: result=z<y >x/////result=0;

////////////////////////////////////////////////////////////////////////////////